FTP Client and Server — Lab Exam Algorithms

FTP Server — Algorithm

1. Start the server program.

2. Create a `ServerSocket` using port `1187` and wait for a client connection.

```
ServerSocket server = new ServerSocket(1187);
Socket socket = server.accept();
```

3. Create a `BufferedReader` object to read the file name entered by the server user.

```
BufferedReader keyboard =
    new BufferedReader(
        new InputStreamReader(System.in));
```

4. Create a `PrintWriter` object to send file contents to the client.

```
PrintWriter out =
    new PrintWriter(
        socket.getOutputStream(), true);
```

5. Ask the server user to enter the name of the text file.

```
System.out.print("Enter the text file name: ");
String filename = keyboard.readLine();
```

6. Create a `File` object using the given file name.

```
File file = new File(filename);
```

7. Check whether the file exists.

```
if (file.exists())
```

8. If the file exists, open the file for reading.

```
BufferedReader in =
    new BufferedReader(
        new FileReader(file));
```

9. Read the file line by line and send each line to the client.

```
while ((line = in.readLine()) != null)
{
    out.println(line);
}
```

10. Close the file after sending its contents.

```
in.close();
```

11. Display a message indicating that the file was sent successfully.

```
System.out.println("The file sent successfully");
```

12. If the file does not exist, display `"File not found"`.

```
else
{
    System.out.println("File not found");
}
```

13. Close the client socket and server socket.

```
socket.close();
server.close();
```

14. Stop the program.

FTP Client — Algorithm

1. Start the client program.

2. Create a `Socket` and connect to the server running on `localhost` at port `1187`.

```
Socket socket =
    new Socket("localhost", 1187);
```

3. Create a `BufferedReader` object to receive the file contents from the server.

```
BufferedReader in =
    new BufferedReader(
```

```
        new InputStreamReader(
            socket.getInputStream()));
```

4. Display a message indicating that the file contents will be displayed.

```
System.out.println(
    "The content of the file is:");
```

5. Read the file contents line by line from the server.

```
while ((line = in.readLine()) != null)
{
    System.out.println(line);
}
```

6. Display a message indicating that the file has been received successfully.

```
System.out.println(
    "The file is received successfully");
```

7. Close the client socket.

```
socket.close();
```

8. Stop the program.